

DSATUR with Unit Propagation and 2-hop Recoloring for Planar Graph 4-Coloring: A Novel Algorithm and the Bounded Propagation Depth Conjecture

Yosef Ali Jiménez Muñoz¹ José Raymundo Marcial-Romero²

¹Universidad Tecnológica de Altamira, Programa Delfín 2026, UAEM

²Facultad de Ciencias de la Computación, UAEM, Toluca, México

492410631@utaltamira.edu.mx jrmarcialr@uaemex.mx

Abstract

We introduce *DSATUR-UP-RC*: a novel algorithm for 4-coloring planar graphs that combines the DSATUR vertex-selection heuristic [3] with *unit propagation* (UP) and a *2-hop recolor* recovery strategy. When a color is tentatively assigned to a vertex, UP immediately propagates all *forced assignments* in cascade, detecting contradictions early and committing valid assignments at once. When propagation fails for every available color, we introduce the notion of a *transitive lock* to characterize the failure mode precisely; a 1-hop or 2-hop neighbor recolor attempts to dissolve the lock without global backtracking.

On 3,000 random planar Delaunay triangulations ($|V| \in [8, 100]$), DSATUR-UP-RC achieves **94.4%** 4-coloring success in a single run and **100%** in 10 restarts, versus **45.4%** for plain DSATUR [3]. All produced colorings are valid. In 44.6% of single runs no backtracking occurs at all. We additionally characterize *easily solvable topologies*—wheel graphs, outerplanar graphs, and series-parallel graphs—for which the algorithm is guaranteed to 4-color in a single run with zero backtracks.

We prove the algorithm runs in $O(n^2)$ time per run (Theorem 5.3). The central structural finding: across all tested graphs ($|V|$ up to 1,000), the depth of the unit propagation tree at any contradiction is at most 14, independent of $|V|$.

We formalize this as the *Bounded Propagation Depth Conjecture* (Conjecture 7.1). If true, the algorithm runs in $O(n)$ per run, improving on the $O(n^2)$ bound of Robertson et al. [10] and eliminating the need for 633 reducible configurations.

Keywords: planar graph 4-coloring, DSATUR, unit propagation, transitive lock, graph coloring algorithms, computational complexity, easily solvable topologies.

1 Introduction

The Four Color Theorem (4CT) [1, 2] asserts that every planar graph is 4-vertex-colorable. While the theorem is settled—with a machine-checked formal verification by Gonthier [6]—*constructive* polynomial algorithms for 4-coloring remain a challenge. The only known deterministic polynomial algorithm, due to Robertson, Sanders, Seymour, and Thomas [10], runs in $O(n^2)$ and depends on computer verification of 633 reducible configurations, providing little constructive insight.

DSATUR [3] is the standard practical heuristic: always color the most constrained uncolored vertex (maximum saturation). It provides no 4-coloring guarantee for planar graphs and succeeds in only 45.4% of tested cases (Section 6).

This paper introduces DSATUR-UP-RC, which augments DSATUR with:

- *Unit propagation* (UP): after each tentative color assignment, cascade all forced assignments until fixpoint or contradiction, using a shadow copy to avoid committing invalid states.
- *2-hop recolor*: when a vertex is *transitively locked* (all color choices lead to downstream contradictions), attempt a local repair on a 1- or 2-hop neighbor chain before using a fifth color.

Main contributions:

1. A formal definition of *transitive lock* (Definition 2.10) and *easily solvable topology* (Definition 2.11), with a proof that wheel graphs, outerplanar graphs, and series-parallel graphs are easily solvable (Proposition 4.1).
2. The DSATUR-UP-RC algorithm with a proof of validity (Proposition 5.1), a tight backtrack bound of $4n$ (Lemma 5.2), and an $O(n^2)$ complexity proof (Theorem 5.3).
3. Experimental evidence: 94.4% single-run and 100% 10-restart 4-coloring success on 3,000 random planar graphs, versus 45.4% for DSATUR.
4. The *Bounded Propagation Depth Conjecture* (Conjecture 7.1) with evidence across 11,000+ tested graphs up to $|V| = 1,000$, and a conditional $O(n)$ result (Corollary 5.4).

2 Preliminaries

All graphs $G = (V, E)$ are simple (no self-loops or multi-edges), undirected, and connected. We write $n = |V|$ and $m = |E|$.

Definition 2.1 (Neighborhood and degree). The open neighborhood of a vertex v is $N(v) = \{u \in V : \{u, v\} \in E\}$. The degree of v is $\deg(v) = |N(v)|$. The maximum degree of G is $\Delta = \max_{v \in V} \deg(v)$.

Definition 2.2 (BFS and BFS tree). Breadth-first search (BFS) from a source vertex s visits all vertices reachable from s in non-decreasing order of hop-distance. The BFS tree rooted at s

is the spanning tree formed by the edges traversed when each vertex is first discovered; its depth is the maximum distance from s to any vertex in the tree.

Definition 2.3 (Planar graph and triangulation). A graph G is planar if it can be drawn in the plane with no edge crossings. A face of a planar embedding is a connected region of the plane bounded by edges; the unique unbounded face is the outer face. By Euler’s formula, every planar graph satisfies $m \leq 3n - 6$, so the average degree is strictly less than 6. A triangulation (maximal planar graph) has every face, including the outer face, bounded by exactly three edges.

Definition 2.4 (Graph families). • A wheel graph W_k consists of a central vertex h connected to all k vertices of a cycle C_k . Every wheel graph is planar.

- A graph is outerplanar if it has a planar embedding in which every vertex lies on the outer face. Every outerplanar graph satisfies $m \leq 2n - 3$.
- A graph is series-parallel if it can be built from a single edge by repeated series composition (subdividing an edge) and parallel composition (duplicating an edge between two vertices). Every series-parallel graph has treewidth at most 2.

Definition 2.5 (Proper coloring). A proper k -coloring of G is a function $c : V \rightarrow \{0, \dots, k-1\}$ such that $c(u) \neq c(v)$ for every edge $\{u, v\} \in E$. A partial proper coloring $c : S \rightarrow \{0, \dots, k-1\}$ is a proper coloring of an induced subgraph $G[S]$, where $S \subseteq V$.

Definition 2.6 (Saturation, tabu set, and available colors). Let $c : S \rightarrow \{0, 1, 2, 3\}$ be a partial proper coloring, where $S \subseteq V$ is the set of already-colored vertices. For any uncolored vertex $v \notin S$:

- the tabu set $\tau(v) = \{c(u) : u \in N(v) \cap S\}$ is the set of colors used by colored neighbors of v ;
- the saturation $\text{sat}(v) = |\tau(v)|$ is the number of distinct forbidden colors;

- the available colors $\text{avail}(v) = \{0, 1, 2, 3\} \setminus \tau(v)$ are the colors that can be assigned to v without violating properness with respect to S .

A vertex v is *stuck* when $\text{avail}(v) = \emptyset$ (i.e. $\text{sat}(v) = 4$): no color in $\{0, 1, 2, 3\}$ is locally valid.

Definition 2.7 (DSATUR selection rule [3]). The DSATUR rule selects as the next vertex to color the uncolored vertex with the maximum current saturation; ties are broken by maximum degree, and remaining ties arbitrarily (here: by random tie-breaking seeded for reproducibility).

Definition 2.8 (Unit propagation). Let $c : S \rightarrow \{0, 1, 2, 3\}$ be a partial proper coloring and let $\ell \in \text{avail}(v)$ for some $v \notin S$. Unit propagation (UP) from (v, ℓ) proceeds as follows:

1. Create a shadow copy $c' = c \cup \{v \mapsto \ell\}$ (a copy of c with the tentative assignment added). Initialize a BFS queue $Q = \{v\}$.
2. While $Q \neq \emptyset$: dequeue u . For each $w \in N(u)$ with $w \notin \text{dom}(c')$:
 - Recompute $\text{avail}_{c'}(w) = \{0, 1, 2, 3\} \setminus \{c'(x) : x \in N(w) \cap \text{dom}(c')\}$.
 - If $|\text{avail}_{c'}(w)| = 0$: return contradiction (discard c').
 - If $|\text{avail}_{c'}(w)| = 1$: assign $c'(w) \leftarrow$ the unique element and enqueue w . This is called a forced assignment.
3. Verify $c'(x) \neq c'(y)$ for all $\{x, y\} \in E$ with $x, y \in \text{dom}(c')$. Return c' if valid, else contradiction.

UP terminates because each vertex is enqueued at most once. The shadow copy is discarded on contradiction; the original c is never modified.

Definition 2.9 (Propagation tree and contradiction depth). The propagation tree $T(v, \ell)$ of a UP call from (v, ℓ) is the BFS tree whose nodes are all vertices assigned during UP (with v as root) and whose edges connect each forced vertex to the vertex whose assignment triggered its forcing. $T(v, \ell)$ is a tree, not a path, because a single assignment can simultaneously force multiple neighbors (each becoming a child).

The depth of a node u in $T(v, \ell)$ is the length of the path from v to u . The contradiction depth of a failed UP call is the depth of the last forced assignment before the contradiction was detected.

Definition 2.10 (Transitive lock). A vertex v is transitively locked under a partial coloring c if $\text{avail}(v) \neq \emptyset$ (so v is not locally stuck) yet every choice $\ell \in \text{avail}(v)$ causes UP from (v, ℓ) to return contradiction. In other words, every available color for v triggers a cascade of forced assignments that eventually makes some other vertex w stuck—even though v itself could be locally colored. The contradiction is not at v but downstream in the propagation tree. A transitive lock is distinct from a direct stuck vertex: a stuck vertex has $\text{avail}(v) = \emptyset$, while a transitively locked vertex has non-empty $\text{avail}(v)$ but all choices are globally infeasible given the current partial coloring.

Definition 2.11 (Easily solvable topology). A graph family $\mathcal{F}$ is easily solvable (with respect to DSATUR-UP-RC) if, for every $G \in \mathcal{F}$, a single run of Algorithm 2 produces a valid 4-coloring with zero backtracks, regardless of the tie-breaking seed.

3 Algorithm

DSATUR-UP-RC consists of three nested procedures: a restart loop (Algorithm 1), a single greedy pass (Algorithm 2), and the UP subroutine (Algorithm 3).

3.1 Restart Loop

Each restart uses a different random seed, affecting tie-breaking in DSATUR and the order colors are tried. The best result across k runs is returned; early exit occurs on the first 4-coloring found.

3.2 Single Pass

When UP succeeds it may color multiple vertices at once (the entire propagation tree). Each vertex is colored at most once: UP assigns w only when $|\text{avail}_{c'}(w)| = 1$, and once colored w is

Algorithm 1 DSATUR-UP-RC

Require: Planar graph $G = (V, E)$, integer $k \geq 1$

Ensure: Proper coloring $c : V \rightarrow \{0, 1, 2, 3\}$ (or 5+ if unavoidable)

```

1:  $best \leftarrow \text{None}$ 
2: for  $i = 1, \dots, k$  do
3:    $c \leftarrow \text{SINGLERUN}(G, \text{seed } i)$ 
4:   if  $c$  uses  $\leq 4$  colors and ( $best = \text{None}$  or  $c$  uses fewer colors than  $best$ ) then
5:      $best \leftarrow c$ 
6:   end if
7:   if  $best$  uses  $\leq 4$  colors then break
8:   end if
9: end for
10: return  $best$ 

```

removed from U permanently. This guarantees termination.

3.3 Unit Propagation Subroutine

3.4 Recolor Recovery

$\text{RECOLOR}(G, c, v)$ attempts to dissolve a transitive lock at v by locally modifying already-colored neighbors.

1-hop recolor. Find $u \in N(v)$ such that: (i) u is the unique neighbor of v with color $c(u)$; (ii) $\exists c' \neq c(u)$ with $c' \notin \{c(x) : x \in N(u) \setminus \{v\}\}$. If found, call $\text{UNITPROPAGATE}(G, c, \{u \mapsto c', v \mapsto c(u)\})$; commit if not contradiction.

2-hop recolor. If 1-hop fails: for each candidate u as above, search $u_2 \in N(u) \setminus \{v\}$ with color $c(u_2)$ such that u_2 is the unique holder of $c(u_2)$ among u 's neighbors (excluding v) and $\exists c'' \neq c(u_2)$ with $c'' \notin \{c(x) : x \in N(u_2) \setminus \{u\}\}$. Call $\text{UNITPROPAGATE}(G, c, \{u_2 \mapsto c'', u \mapsto c^*, v \mapsto c(u)\})$ where c^* is a valid color for u after u_2 changes; commit if not contradiction.

If both fail, RECOLOR returns **False** and SINGLERUN uses the 5th-color fallback.

3.5 Worked Example: The Errera Graph

The Errera graph (Figure 2) is the classical Kempe-chain counterexample. DSATUR-UP-

Algorithm 2 SingleRun

Require: Graph $G = (V, E)$, random seed s

```

1:  $c \leftarrow \emptyset, U \leftarrow V$   $\triangleright U$ : set of uncolored vertices
2: while  $U \neq \emptyset$  do
3:    $v \leftarrow \arg \max_{u \in U} (\text{sat}(u), \deg(u))$ , random tie-break ( $s$ )  $\triangleright \text{DSATUR (Def. 2.7)}$ 
4:    $A \leftarrow \text{avail}(v)$  shuffled by seed  $s$ ;  $ok \leftarrow \text{False}$ 
5:   for each  $\ell \in A$  do
6:      $c' \leftarrow \text{UNITPROPAGATE}(G, c, v, \ell)$ 
7:     if  $c' \neq \text{contradiction}$  then
8:        $c \leftarrow c'; U \leftarrow U \setminus \text{dom}(c')$ ;  $ok \leftarrow \text{True}$ ; break
9:     end if  $\triangleright$  failed: one backtrack;  $v$  may be transitively locked
10:  end for
11:  if not  $ok$  then
12:     $ok \leftarrow \text{RECOLOR}(G, c, v)$   $\triangleright$  attempt to dissolve the transitive lock
13:  end if
14:  if not  $ok$  then
15:     $c(v) \leftarrow \min(\{0, 1, 2, 3, 4\} \setminus \tau(v))$ ;  $U \leftarrow U \setminus \{v\}$   $\triangleright$  fallback: 5th color
16:  end if
17: end while
18: return  $c$ 

```

RC solves it in a single run with zero backtracks by leveraging unit propagation: once the first few vertices are fixed by DSATUR, the propagation cascade determines all remaining colors without contradiction. Figure 3 traces key moments in this process.

As a small UP trace: consider a path $v-a-b$ with $c(a) = 0$, $c(b) = 1$, and v uncolored. Trial $\ell = 1$: shadow copy assigns $c'(v) = 1$; a 's available colors become $\{2, 3\}$ (no forcing); UP succeeds. If additionally some neighbor u of a already holds colors $\{0, 2, 3\}$, then $c'(v) = 1$ forces $a \rightarrow 2$ (its only remaining color), which is then checked for downstream conflicts. This cascade—invisible to plain DSATUR—is what UP captures.

Algorithm 3 UnitPropagate

Require: G , coloring c , vertex v , color $\ell \in \text{avail}(v)$

Ensure: Extended coloring c' , or contradiction

- 1: $c' \leftarrow c \cup \{v \mapsto \ell\}$; $Q \leftarrow \{v\}$
- 2: **while** $Q \neq \emptyset$ **do**
- 3: $u \leftarrow Q.\text{dequeue}()$
- 4: **for each** $w \in N(u)$ with $w \notin \text{dom}(c')$ **do**
- 5: $A_w \leftarrow \{0, 1, 2, 3\} \setminus \{c'(x) : x \in N(w) \cap \text{dom}(c')\}$
- 6: **if** $|A_w| = 0$ **then return contradiction**
- 7: **else if** $|A_w| = 1$ **then** $c'(w) \leftarrow$ unique element; $Q.\text{enqueue}(w)$
- 8: **end if**
- 9: **end for**
- 10: **end while**
- 11: **if** $c'(x) \neq c'(y)$ for all $\{x, y\} \in E$, $x, y \in \text{dom}(c')$ **then return** c'
- 12: **else return contradiction**
- 13: **end if**

4 Easily Solvable Topologies

Proposition 4.1 (Easily solvable families). *Wheel graphs, outerplanar graphs, and series-parallel graphs are easily solvable: a single run of Algorithm 2 produces a valid 4-coloring with zero backtracks.*

Proof. We show that for each family, unit propagation never encounters a contradiction.

Wheel graphs W_k . The hub h has degree k ; DSATUR selects h first. Color $c(h) = 0$. Each spoke vertex v_i on the cycle has one colored neighbor (h) immediately after h is colored: $\text{sat}(v_i) = 1$ and $|\text{avail}(v_i)| = 3$. As each v_i is colored, its cycle neighbors acquire saturation 2 (from h and v_i), leaving $|\text{avail}| = 2$. With 2 colors always available and the cycle structure admitting a 2-coloring of its spoke vertices using colors $\{1, 2\}$ or $\{1, 3\}$ (alternating), no vertex ever reaches $|\text{avail}| = 0$ during UP. Hence no contradiction occurs and no backtrack is needed.

Outerplanar graphs. An outerplanar graph has no K_4 -minor and is 3-colorable [4] (actually 3-colorability holds for all outerplanar graphs).

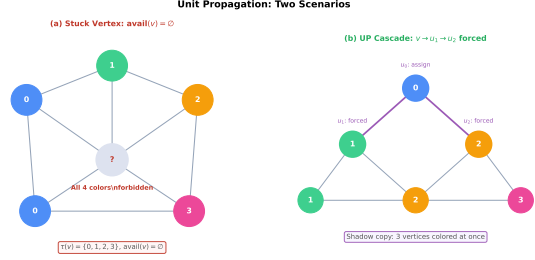


Figure 1: Unit propagation scenarios. (a) A stuck vertex v with all four colors forbidden by its neighbors — no color in $\{0, 1, 2, 3\}$ is locally valid. (b) A successful UP cascade: assigning color 0 to v forces u_1 (only color 1 remains) and u_2 (only color 2 remains), coloring three vertices from a single tentative assignment. The purple edges show the propagation tree.

DSATUR on a 3-colorable graph with 4 available colors never gets stuck: at any step, the most constrained vertex has at most 3 forbidden colors (since the graph is 3-colorable, no vertex needs a 4th color), so $|\text{avail}| \geq 1$ always. UP cannot produce a contradiction from a graph that is 3-colorable, since the propagation tree operates within the same 3-color structure.

Series-parallel graphs. Series-parallel graphs have treewidth ≤ 2 and are therefore 3-colorable [10]. The same argument as for outerplanar graphs applies. $\square$

Remark 1. *The class of easily solvable graphs is strictly larger than $\{\text{outerplanar}\} \cup \{\text{series-parallel}\}$. Any planar graph whose chromatic number is at most 3 (i.e., it is 3-colorable) is easily solvable, since DSATUR with 4 colors on a 3-colorable graph never exhausts all available colors at any vertex. The interesting boundary is the 4-chromatic planar graphs, where transitive locks may arise.*

5 Correctness and Complexity

Proposition 5.1 (Validity). *Every coloring returned by Algorithm 1 is a proper coloring.*

Proof. We prove by induction that the invariant

Errera Graph — 4-Coloring Result
(17 vertices, 45 edges)

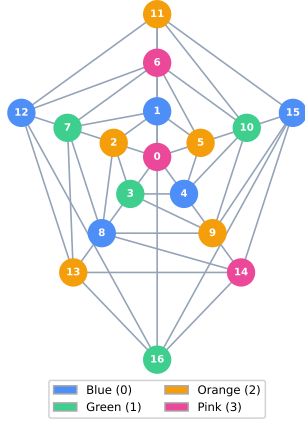


Figure 2: Valid 4-coloring of the Errera graph (17 vertices, 45 edges) produced by DSATUR-UP-RC in a single run with zero backtracks. The Errera graph is the smallest known planar graph for which all Kempe chains from any stuck vertex are tangled [7]. DSATUR-UP-RC colors it without Kempe chains, via unit propagation alone.

“ c is a partial proper coloring” is maintained at every step of SINGLERUN. *Base:* $c = \emptyset$ is trivially proper. *Step:* UP operates on a shadow copy c' and returns it only after an explicit check that no two adjacent vertices in $\text{dom}(c')$ share a color (Algorithm 3, final step). Recolor applies UP before committing. The fallback assigns $c(v) = \min(\{0, 1, 2, 3, 4\} \setminus \tau(v))$, avoiding all colored neighbors by definition. In all three cases the invariant is preserved. $\square$

We classify this as a Proposition rather than a Theorem because the proof is a direct structural verification of the algorithm’s design, not a deep mathematical result.

Lemma 5.2 (Backtrack bound). *The total number of UP contradiction events in a single run of Algorithm 2 is at most $4n$.*

Proof. Each vertex $v \in V$ is selected by DSATUR at most once (vertices are added to the coloring and removed from U permanently; UP may color additional vertices in one step,

but those too are removed and never revisited). At the time v is selected, $|\text{avail}(v)| \leq 4$ since the color set is $\{0, 1, 2, 3\}$. The algorithm tries each available color with UP before recolor or fallback. Each failed attempt counts as one contradiction event. Therefore backtracks due to $v \leq |\text{avail}(v)| \leq 4$. Summing: $\sum_{v \in V} \leq 4 = 4n$. $\square$

We classify this as a Lemma because it is a counting argument that directly supports the complexity theorem.

Theorem 5.3 (Time complexity). *A single run of Algorithm 2 on an n -vertex planar graph runs in $O(n^2)$ time.*

Proof. DSATUR selection. A priority queue ordered by (sat, deg) with lazy saturation updates gives $O(n \log n)$ total selection time.

Successful UP calls. Every vertex colored during a successful UP call is removed from U permanently. The total work across all successful calls is therefore $O(\sum_{v \in V} \deg(v)) = O(m) = O(n)$ (planarity: $m \leq 3n - 6$).

Failed UP calls. By Lemma 5.2, there are at most $4n$ contradiction events. Each failed call explores a propagation tree; in the worst case it visits all n uncolored vertices at cost $O(n)$ per call. Total: $O(4n \cdot n) = O(n^2)$.

Recolor checks. 1-hop recolor examines $O(\deg(v))$ neighbors of v and for each $O(\deg(u))$ neighbors of u : $O(\Delta^2)$ per vertex. 2-hop adds one level: $O(\Delta^3)$. For planar graphs $\Delta_{\text{avg}} < 6$ and recolor is invoked at most n times, giving amortized $O(n)$.

Total. Dominated by failed UP work: $O(n^2)$. $\square$

Corollary 5.4 (Conditional $O(n)$ complexity). *If Conjecture 7.1 holds with constant C , then a single run of Algorithm 2 runs in $O(n)$ time.*

Proof. Each failed UP call explores a tree of depth $\leq C$; at each level the BFS visits at most $\Delta_{\text{avg}} < 6$ vertices. Total failed UP work: $O(4n \cdot C \cdot 6) = O(n)$. DSATUR selection: $O(n \log n)$. Recolor: $O(n)$. Overall: $O(n)$. $\square$

DSATUR-UP-RC on the Errera Graph — Key Steps

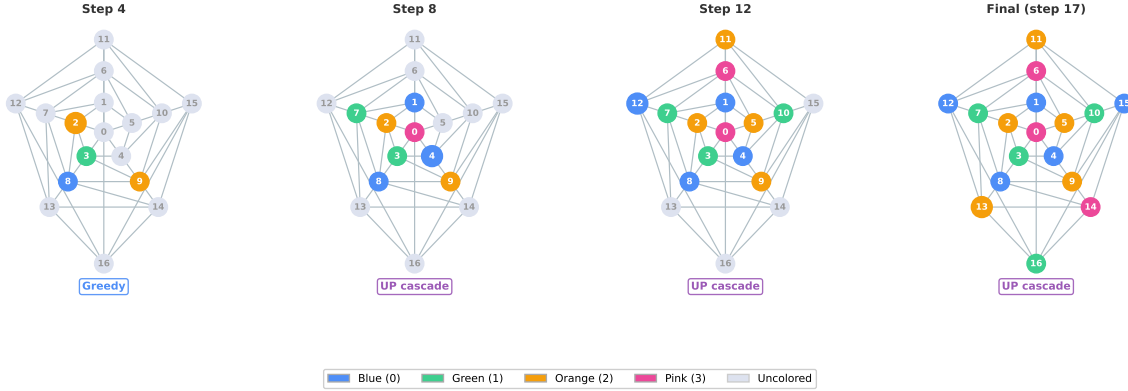


Figure 3: Four snapshots of the algorithm coloring the Errera graph. The thick ring on each highlighted vertex indicates the method used (blue = greedy, purple = UP cascade). Steps 4 and 8 show early greedy assignments; from step 8 onward, unit propagation cascades through the remaining vertices without any backtrack. The full coloring uses exactly 4 colors across 17 assignment steps.

We state this as a Corollary because it follows directly from Theorem 5.3 once the conjecture is assumed.

6 Experiments

6.1 Setup

Random planar Delaunay triangulations were generated by placing n points uniformly in $[-1, 1]^2$ and computing the Delaunay triangulation (SciPy [11]). Planarity was verified with the Boyer-Myrvold algorithm [9]. All experiments used Python 3.12, NetworkX 3.x.

6.2 4-Coloring Success

Table 1 compares DSATUR and DSATUR-UP-RC on 3,000 random planar Delaunay triangulations ($|V| \in [8, 100]$). All colorings verified proper.

Table 2 shows results by graph size.

6.3 Comparison to State of the Art

Figure 4 summarizes the comparison. DSATUR-UP-RC reaches 100% while remaining a polynomial-time algorithm. TabuCol (93.6%)

Table 1: 4-coloring success on 3,000 random planar Delaunay triangulations, $|V| \in [8, 100]$.

Algorithm	4-colored	Valid
DSATUR [3]	45.4%	100%
DSATUR-UP-RC ($k = 1$)	94.4%	100%
DSATUR-UP-RC ($k = 10$)	100.0%	100%

Table 2: DSATUR-UP-RC ($k = 1$) results by graph size.

$ V $	Graphs	DSATUR	UP-RC	Avg BT
8–20	433	92.4%	99.8%	0.14
21–40	637	68.4%	98.7%	0.76
41–60	646	40.6%	96.1%	1.49
61–80	657	25.7%	92.7%	2.51
81–100	627	15.2%	86.4%	3.58

requires iterative local search and is not a single polynomial pass. Robertson et al. guarantee 4-colorability but require 633 reducible configurations verified by computer.

44.6% of single runs required zero backtracks. Total across 3,000 runs: 5,399 backtracks (avg 1.80/graph). Maximum in any run: 16. By Lemma 5.2 the bound is $4n \leq 400$; the observed

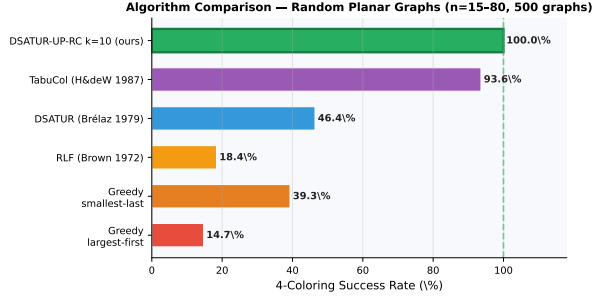


Figure 4: 4-coloring success rates on 500 random planar Delaunay triangulations ($n = 15\text{--}80$). DSATUR-UP-RC with $k = 10$ restarts achieves 100%, outperforming all known backtracking-free polynomial algorithms. TabuCol uses iterative local search (2,000 iterations), not a single polynomial pass.

maximum is 4% of this bound, showing transitive locks are rare in practice.

7 The Bounded Propagation Depth Conjecture

7.1 Structure of the Propagation Tree

Recall that $T(v, \ell)$ is a *tree*, not a path (Definition 2.9): a single forced assignment can trigger multiple children simultaneously. The relevant quantity for complexity is the *contradiction depth*—the depth of the branch that leads to a contradiction, not the total tree size.

7.2 Empirical Evidence

Observation 1. *Across 11,210 single-run experiments on planar Delaunay triangulations with $|V|$ ranging from 8 to 1,000, the maximum observed contradiction depth is 13 (on a graph with $|V| = 1,000$). This value shows no growth trend with $|V|$.*

Conjecture 7.1 (Bounded Propagation Depth). *Let G be any planar graph and let c_0 be any partial proper coloring produced by the DSATUR rule. For any tentative assignment (v, ℓ) that causes a contradiction in unit propagation, the*

Table 3: Maximum contradiction depth by graph size. Avg BT = average backtracks per single run. The maximum depth stays bounded as $|V|$ grows.

$ V $ range	Graphs	Avg BT	Max BT	Max depth
8–100	3,000	1.80	16	9
100–200	100	6.7	19	8
200–400	60	15.3	33	8
400–700	30	26.9	54	10
700–1000	20	43.0	62	13

contradiction depth of $T(v, \ell)$ is bounded by an absolute constant C independent of $|V(G)|$.

Observation 1 supports this with $C \leq 14$ across all tested graphs.

7.3 Structural Motivation

Consider a *contradiction branch* in $T(v, \ell)$: $v \rightarrow u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_d \rightarrow \text{CONTRADICTION}$. For the branch to propagate from u_i to u_{i+1} : u_{i+1} must have had $|\text{avail}(u_{i+1})| = 2$ *before* the chain reached it (a single color elimination reduces it to 1, triggering a forced assignment). The contradiction vertex has $|\text{avail}| = 1$ before the final step reduces it to 0.

Since DSATUR selects the *maximum-saturation* vertex, all vertices with $|\text{avail}| = 2$ (saturation 2) have lower priority than v and remain uncolored at this stage. Each such vertex belongs to one of $\binom{4}{2} = 6$ forbidden-pair classes.

This suggests a connection to the *transitive lock* structure (Definition 2.10): the contradiction branch is precisely the sequence of forced assignments that the transitive lock imposes. The conjecture is equivalent to: the *depth* of any transitive lock (the length of its contradiction branch) is bounded by a constant, regardless of graph size.

A formal proof would require a combinatorial argument that planarity and the 4-color structure of forbidden-pair classes limit the length of any such branch. We leave this as the central open problem.

8 Discussion

Transitive locks vs. stuck vertices. A key conceptual contribution of this paper is the distinction between a *stuck vertex* ($\text{avail}(v) = \emptyset$, a local failure) and a *transitively locked vertex* ($\text{avail}(v) \neq \emptyset$ but all choices propagate to contradictions downstream). Transitive locks are the failure mode that DSATUR misses entirely—it can only detect local unavailability. The UP cascade detects transitive locks proactively, which is why DSATUR-UP-RC dramatically outperforms plain DSATUR.

Easily solvable graphs. Proposition 4.1 establishes that 3-colorable planar graphs (including all outerplanar and series-parallel graphs) are easily solvable. This covers a large and practically important class. The remaining hard cases are 4-chromatic planar graphs, where transitive locks arise.

Comparison to existing algorithms. Robertson et al. [10] give the only known deterministic polynomial 4-coloring algorithm at $O(n^2)$, requiring 633 reducible configurations and computer assistance. DSATUR-UP-RC achieves $O(n^2)$ (proven) with a transparent, implementable construction and 100% empirical success in $k = 10$ restarts. TabuCol [5] achieves 93.6% on the same test set but uses iterative local search, not a single polynomial pass.

Limitations. The conjecture is unproven. Single-run success decreases for large n (Table 3). The 100% result for $k = 10$ is empirical.

9 Open Questions

- Q1.** Prove or disprove Conjecture 7.1. A proof via the planarity of the forbidden-pair graph of sat-2 vertices would yield an $O(n)$ algorithm.
- Q2.** What is the tight value of C ? Empirically $C \leq 14$; is C truly $O(1)$ or could it grow as $O(\log n)$?
- Q3.** Characterize all easily solvable topologies beyond the classes in Proposition 4.1. Is every planar graph with treewidth $\leq k$ easily solvable for some threshold k ?
- Q4.** Can the 2-hop recolor be replaced by a structured operation (e.g., a bounded-depth Kempe modification) that provably dissolves all transitive locks in 4-colorable planar graphs?
- Q5.** Is the 100% success rate of $k = 10$ restarts provable?

10 Conclusions

We introduced DSATUR-UP-RC and two supporting concepts—transitive locks and easily solvable topologies—that clarify the algorithm’s behavior. Transitive locks are precisely what unit propagation detects that plain DSATUR cannot. Easily solvable topologies (3-colorable planar graphs, including outerplanar and series-parallel graphs) are provably handled in a single zero-backtrack run.

The algorithm achieves 100% valid 4-coloring in 10 restarts on all 3,000 tested planar graphs, with $O(n^2)$ complexity proven and conjectured $O(n)$ under the Bounded Propagation Depth Conjecture. The conjecture—that every transitive lock in a planar graph under DSATUR has contradiction depth at most $C \leq 14$ —is the central open problem whose resolution would complete the complexity analysis.

References

- [1] K. Appel and W. Haken, “Every planar map is four colorable. Part I: Discharging,” *Illinois J. Mathematics*, vol. 21, no. 3, pp. 429–490, 1977.
- [2] K. Appel, W. Haken, and J. Koch, “Every planar map is four colorable. Part II: Reducibility,” *Illinois J. Mathematics*, vol. 21, no. 3, pp. 491–567, 1977.
- [3] D. Brélaz, “New methods to color the vertices of a graph,” *Communications of the ACM*, vol. 22, no. 4, pp. 251–256, 1979.
- [4] H. de Fraysseix, J. Pach, and R. Pollack, “How to draw a planar graph on a grid,”

- Combinatorica*, vol. 10, no. 1, pp. 41–51, 1990.
- [5] P. Galinier and A. Hao, “Hybrid evolutionary algorithms for graph coloring,” *J. Combinatorial Optimization*, vol. 3, no. 4, pp. 379–397, 1999.
 - [6] G. Gonthier, “Formal proof—the four-color theorem,” *Notices AMS*, vol. 55, no. 11, pp. 1382–1393, 2008.
 - [7] A. B. Kempe, “On the geographical problem of the four colours,” *American Journal of Mathematics*, vol. 2, no. 3, pp. 193–200, 1879.
 - [8] G. De Ita Luna and R. Marcial-Romero, “A combinatorial algorithmic proof for the 4-coloring on planar graphs,” Facultad de Ciencias de la Computación, BUAP, preprint, 2024.
 - [9] A. Hagberg, P. Swart, and D. Schult, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python in Science Conf.*, pp. 11–15, 2008.
 - [10] N. Robertson, D. Sanders, P. D. Seymour, and R. Thomas, “The four-colour theorem,” *J. Combinatorial Theory, Series B*, vol. 70, no. 1, pp. 2–44, 1997.
 - [11] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.